



Security Audit Report

CoinList TokenSwap

v1.0

July 1, 2026

Table of Contents

Table of Contents

License

Disclaimer

Introduction

Purpose of This Report

Codebase Submitted for the Audit

Methodology

Functionality Overview

How to Read This Report

Code Quality Criteria

Summary of Findings

Detailed Findings

1. Pause or unpause have no access control
2. Adapter flow is incompatible with Superstate DIP
3. Zero-fee partial fills can strand user funds
4. Residual approvals can block future swaps
5. Adapter authorization is not enforced during swaps
6. Preview overstates output when fees are enabled
7. Local DIP interfaces may not match production ABI
8. Missing reentrancy protection in `swap`
9. Adapter does not enforce minimum output locally
10. Swap transactions without a deadline can execute under stale assumptions
11. Fee updates do not emit an event
12. The `Swapped` event underreports the gross user spend when fees are enabled
13. Non-standard payment tokens break swap execution and accounting
14. Swap fee can be set to 100 percent

License



THIS WORK IS LICENSED UNDER A [CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES
4.0 INTERNATIONAL LICENSE](https://creativecommons.org/licenses/by-nc/4.0/).

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED "AS IS", WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUCT ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

<https://oaksecurity.io>

info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by Amalgamated Token Services Inc. to perform a security audit of the CoinList Token Swap Contracts.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	URL	Commit
solidity	https://github.com/coinlist/solidity	df0eed7e7564e0705765e39f299e73817198a101

Scope	<pre>src/ superstate/ IDip.sol IDippable.sol Superstate.sol swap/ ITokenSwap.sol TokenSwap.sol Types.sol</pre>
-------	--

Fixes Verified at Commit

d92da9f3d372312fe8b98e867026a2fd444d0ca3

Note that only fixes to the issues described in this report have been reviewed at this commit.

Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
 - a. Race condition analysis
 - b. Under-/overflow issues
 - c. Key management vulnerabilities
4. Report preparation.

Functionality Overview

TokenSwap.sol defines the shared base for token swap integrations. It stores the output token, swap identifier, fee basis points, and per-user/per-token swap totals. It exposes balance views, fee calculation, swap total views, owner-controlled fee configuration, and owner withdrawals of tokens held by the contract. The actual authorization, preview, and swap behavior are left to child contracts.

Superstate.sol is a concrete TokenSwap implementation for a Superstate “buy the dip” market. It checks user eligibility through the Superstate output token, previews expected spend/output through the current Dip contract, pulls the input token from the caller, optionally deducts a fee, approves the Superstate token contract, calls `buyTheDip`, measures the minted output token balance, transfers the minted output to the caller, and records swap totals.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or losses, with medium to high probability.
Minor	A violation of best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects such as user experience or readability. This category may also include opinionated recommendations that the project team might not agree with.

The **status** of an issue can be one of the following: **Resolved**, **Acknowledged**, or **Partially Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	Low-Medium	–
Code readability and clarity	Medium-High	–
Level of documentation	Medium-High	Client provided extensive documentation.
Test coverage	High	Test coverage computed with forge coverage is 100%

Summary of Findings

No	Description	Severity	Status
1	Pause or unpause have no access control	Critical	Resolved
2	Adapter flow is incompatible with Superstate DIP	Major	Acknowledged
3	Zero-fee partial fills can strand user funds	Major	Resolved
4	Residual approvals can block future swaps	Minor	Resolved
5	Adapter authorization is not enforced during swaps	Minor	Resolved
6	Preview overstates output when fees are enabled	Minor	Resolved
7	Local DIP interfaces may not match production ABI	Minor	Resolved
8	Missing reentrancy protection in swap	Minor	Resolved
9	Adapter does not enforce minimum output locally	Minor	Resolved
10	Swap transactions without a deadline can execute under stale assumptions	Informational	Acknowledged
11	Fee updates do not emit an event	Informational	Resolved
12	The Swapped event underreports the gross user spend when fees are enabled	Informational	Resolved
13	Non-standard payment tokens break swap execution and accounting	Informational	Resolved
14	Swap fee can be set to 100 percent	Informational	Resolved

Detailed Findings

1. Pause or unpause have no access control

Severity: Critical

- `lib/shared/Stopable.sol:12-20`
- `src/swap/TokenSwap.sol`
- `src/superstate/Superstate.sol:32`

The `stop` function in `Stopable.sol` is declared `public virtual` with no access control modifier. `TokenSwap` inherits `Stopable` without overriding `stop`, and `Superstate` inherits `TokenSwap` without overriding it either. The `swap` function in `Superstate` is gated by the `started` modifier with `SWAP_LEVEL` set to 2, which checks that the `stopped` bitmask does not have bit 2 set.

Because `stop` is unrestricted, any address can call it with level 2 to set `stopped` to 2, which causes every subsequent `swap` call to revert with `IsStopped`. The same attacker can immediately call `stop(0)` to undo a legitimate owner pause, then call `stop(2)` again, making the denial of service persistent and impossible to cure without a code upgrade.

The `IStopable` interface documents that `stop` should be "available to owner only". The sibling `TokenSaleFund` contract overrides `stop` with the `onlyOwner` modifier, confirming that the intended pattern was never applied to `TokenSwap`.

Recommendation

We recommend overriding `stop` in `TokenSwap` with the `onlyOwner` modifier, mirroring the `TokenSaleFund` implementation. This single override restores access control for all contracts in the inheritance chain and eliminates the unrestricted pause and unpause vectors.

Status: Resolved

2. Adapter flow is incompatible with Superstate DIP

Severity: Major

- `src/superstate/Superstate.sol:76`
- `src/superstate/IDippable.sol:16`
- `test/superstate/integration/Superstate.t.sol:122-123`

The adapter executes a `Superstate` purchase by having `swap` call `buyTheDip` on the configured `outputToken`. That makes the adapter, not the end user, the direct caller of the `Superstate EquityToken`. `Superstate's` Direct Issuance Program documentation describes `buyTheDip` as using

`msg.sender` as the buyer, pulling payment tokens from that address and minting equity tokens back to it. The same documentation says router and proxy contracts are unsupported, with partners expected to send transactions directly from user wallets.

Under the adapter design, Superstate evaluates `msg.sender` as the adapter address. Compliance checks, purchase attribution, and any emitted Superstate records therefore refer to the adapter rather than the individual buyer. The integration tests reflect this exact setup by allowlisting the adapter contract and checking `authorized` for the adapter instead of the user's address.

Once the adapter is allowlisted, any wallet can route purchases through it regardless of that wallet's own allowlist status. This bypasses user-level compliance enforcement. If production Superstate follows the documented no-router constraint strictly, the integration may fail outright rather than merely attributing purchases to the adapter.

Recommendation

We recommend confirming with Superstate whether an intermediary-compatible API exists or is planned, ideally one that accepts explicit buyer and recipient addresses. If such an API exists, align the local interfaces and tests with it. If it does not, the adapter architecture should be redesigned because the documented DIP flow does not support contract-mediated purchases.

Status: Acknowledged

The client acknowledges the issue stating that the apparent incompatibility arises from incomplete public documentation on Superstate's part regarding contract-mediated purchases. Per direct discussions with Superstate, whitelisting the integration contract by hand is an agreed-upon and supported practice. All parties have aligned on this approach, and the finding therefore does not represent an actual risk in the production context of this integration.

3. Zero-fee partial fills can strand user funds

Severity: Major

- `src/superstate/Superstate.sol:32-91`

Superstate documents that `buyTheDip` can fill less than the requested purchase when market capacity is lower than the requested amount. In the native user-to-EquityToken flow, this is safe because the token contract pulls only the amount it actually uses from the buyer.

The adapter changes that flow by taking custody of user funds before it calls `Superstate`. In the zero-fee branch, where `bps` is 0, `swap` skips `preview`, transfers the user's full requested amount into the adapter, approves the same full amount to `outputToken`, and then calls `buyTheDip`. If the purchase is partially filled, the unspent payment tokens are left inside the adapter. There is no automatic refund and `inputSum` still records the full requested amount, so storage no longer matches the actual spend. The only recovery route is an owner-managed sweep through `transfer`.

The fee-enabled path behaves differently: when `bps` is nonzero, it calls `preview` and reduces `input` to the available supply before pulling funds. The zero-fee branch omits that sizing step, so the safer behaviour exists only when a fee is configured.

Users can therefore end up with payment tokens trapped in the adapter and no self-service claim mechanism. Because the contract does not track residual balances per user, any later reimbursement requires manual off-chain attribution and owner intervention.

Recommendation

We recommend using `preview` to size the purchase in both fee and no-fee modes, then transferring only the effective payment amount from the caller. Also return any remaining payment-token balance attributable to the call after `buyTheDip` completes, so partial fills cannot strand user funds in the adapter.

Status: Resolved

4. Residual approvals can block future swaps

Severity: Minor

- `src/superstate/Superstate.sol:68`

`swap` grants `outputToken` an allowance for input payment tokens by calling `safeApprove`. Once `buyTheDip` returns, the adapter leaves that allowance untouched. If the external purchase uses less than the approved amount, such as during a documented partial fill, the unused approval remains live with `outputToken` as spender.

This is especially fragile because `swap` accepts an arbitrary token. Tokens with USDT-style approval rules reject changing a nonzero allowance directly to another nonzero allowance. After one partial fill leaves a residual approval, later swaps using that token can revert at the approval step. The codebase already has a safer pattern: `TokenSaleFund` uses `safeApproveWithRetry`, which clears the allowance first when needed.

One partial fill can therefore disable future swaps for a USDT-like input token. Since Superstate has no dedicated approval-reset function, the stuck allowance cannot be repaired directly by the owner through the adapter.

Recommendation

We recommend using `safeApproveWithRetry` instead of `safeApprove` for the approval in `Superstate.sol`. Another acceptable approach is to clear the allowance back to zero after `buyTheDip`, preventing residual approvals from carrying into later swap calls.

Status: Resolved

5. Adapter authorization is not enforced during swaps

Severity: Minor

- `src/superstate/Superstate.sol:22-24`
- `src/superstate/Superstate.sol:32`
- `test/superstate/integration/Superstate.t.sol:122-123`

Superstate includes an authorized helper that reads the `EquityToken` allowlist through `isAllowed`. However, `swap` does not use this helper to validate `msg.sender` before it starts the purchase flow.

The integration tests show the resulting model: the allowlisted address is the adapter, not the user's wallet. That means Superstate sees the adapter as the buyer, so any wallet can submit a purchase through the allowlisted adapter even if that wallet is not itself on the allowlist. Conversely, if the adapter is not allowlisted, even allowlisted users cannot complete swaps through it.

At present, authorization depends on `buyTheDip` rejecting non-allowlisted callers. If a future `IDippable` implementation changes or drops that check, the adapter has no independent enforcement. The presence of `authorized` also suggests to callers that the adapter participates in authorization, even though it is not consulted by `swap`.

Recommendation

We recommend requiring `authorized(msg.sender)` to succeed at the beginning of `swap`, before any funds are pulled. This makes the adapter's exposed authorization API effective and avoids relying entirely on the external purchase contract for that security boundary.

Status: Resolved

6. Preview overstates output when fees are enabled

Severity: Minor

- `src/superstate/Superstate.sol:26-30`
- `src/superstate/Superstate.sol:39-50`

`preview` quotes the Superstate DIP output for the full amount supplied by the caller. The execution path differs when `bps` is nonzero: `swap` first subtracts the fee from the previewed input, then sends only the remainder into `buyTheDip`. As a result, the public quote is based on a larger purchase than the one that actually occurs.

Superstate instructs integrators to derive the minimum output from a `preview` response. With fees enabled, that guidance leads to a bad choice: either the integrator sets `slip` from the overstated quote and the `swap` call reverts, or they loosen `slip` enough to pass and the user receives less than the quoted output.

The public API therefore gives users and integrators a systematically high `swap` estimate whenever fees apply. That breaks safe `slip` selection and can produce either avoidable transaction failures or successful transactions that under-deliver relative to the displayed quote.

Recommendation

We recommend making `preview` follow the same fee math as `swap`: deduct the fee from input first, then quote the net purchase amount. If callers need more detail, add a richer `preview` variant that exposes gross input, fee, net input, and expected output separately.

Status: Resolved

7. Local DIP interfaces may not match production ABI

Severity: Minor

- `src/superstate/IDip.sol:7`
- `src/superstate/IDippable.sol:7`
- `src/superstate/Superstate.sol:26-30`

`preview` is wired to locate the DIP contract by calling `dipContract` on `IDippable`, then request a quote by calling `calculateAmount` on `IDip`. Superstate's published Direct Issuance Program ABI describes a different quote function, `calculateOutput`, with the same apparent parameters, and it does not list `dipContract` as an `EquityToken` accessor.

If the deployed Superstate contracts match the documentation, calls made through the local interfaces will hit missing selectors. In that case, preview cannot succeed, and any swap path with bps greater than zero will fail before reaching the purchase. Tests do not reveal this because MockTheDip implements the local calculateAmount name instead of the documented production name.

Recommendation

We recommend updating IDip.sol to use calculateOutput if that is the production ABI. Separately verify whether dipContract exists on the real EquityToken; if not, remove it from IDippable.sol and use the correct discovery mechanism. The mocks and tests should be changed to the same ABI so selector mismatches are caught locally.

Status: Resolved

8. Missing reentrancy protection in swap

Severity: Minor

- `src/superstate/Superstate.sol:76-88`

The swap function performs an external call to buyTheDip on the configured outputToken in `src/superstate/Superstate.sol:76` and afterward reads balances, transfers output tokens to the caller, and updates the stored totals in lines 78-88. State is written after external calls.

Exploitability depends on the implementations of EquityToken and payment tokens. These are expected to be trusted, but they are external, upgradable contracts outside the control of this codebase.

Recommendation

We recommend adding a reentrancy guard to the swap function as a defense-in-depth measure.

Status: Resolved

9. Adapter does not enforce minimum output locally

Severity: Minor

- `src/superstate/Superstate.sol:76-82`

`swap` passes the user-provided `slip` value into `buyTheDip`, then derives the actual minted amount from the adapter's output-token balance change. After computing that value, the adapter transfers the output without independently requiring `minted >= slip`.

The Superstate Direct Issuance Program documentation says the DIP layer should revert with `InsufficientOutput` when the minimum is not met. Even so, once the adapter has calculated `minted`, it has enough information to enforce the same user guarantee itself. Without that local check, correctness depends entirely on the configured `outputToken` honoring `slip`.

The current mock in `BuyTheDip.t.sol` always mints exactly `slip`, so the tests do not exercise a failing minimum-output case. If an `IDippable` implementation ignores or misapplies the parameter, a caller can pay the full input and still receive less than the minimum they specified, including no equity tokens at all.

Recommendation

We recommend reverting unless `minted` is greater than or equal to `slip` after the balance delta is known. This gives the adapter its own minimum-output enforcement and prevents user protection from depending solely on the external DIP implementation.

Status: Resolved

10. Swap transactions without a deadline can execute under stale assumptions

Severity: Informational

- `src/swap/ITokenSwap.sol:48`
- `src/superstate/Superstate.sol:32`

The `swap` function, declared in `src/swap/ITokenSwap.sol:48` and implemented in `src/superstate/Superstate.sol:32`, accepts a minimum output via `slip` but no execution deadline. A pending transaction stays valid indefinitely and can be mined long after submission, as long as the original `slip` value is still met.

Execution depends on mutable state: Superstate market capacity and quotes change over time, and the owner can change the fee between submission and execution. Users who set `slip` loosely to tolerate normal quote drift may see their transactions execute later than intended, while users who set it tightly face avoidable reverts.

Recommendation

We recommend adding a user-supplied deadline parameter to the `swap` function and reverting when `block.timestamp` exceeds it.

Status: Acknowledged

The client acknowledges the issue stating that they understand the risk of swap transactions executing under stale market conditions without a deadline constraint.

11. Fee updates do not emit an event

Severity: Informational

- `src/swap/TokenSwap.sol:79-85`

The owner can update the fee basis points through `setBps`, but that state transition is silent: no event is emitted when `bps` changes. This is inconsistent with nearby administrative actions, where `Stopable` emits `Stopped` and token movement emits `Transferred`.

Off-chain systems therefore have no event stream to follow for fee updates. Indexers, monitoring jobs, and frontends must poll storage to detect changes, and historical fee changes are not represented in a canonical on-chain log.

Recommendation

We recommend defining and emitting a `BpsSet` event whenever `setBps` updates `bps`. Include both the previous value and the replacement value so off-chain consumers can reconstruct fee history directly from logs.

Status: Resolved

12. The Swapped event underreports the gross user spend when fees are enabled

Severity: Informational

- `src/superstate/Superstate.sol:88`

The swap function emits the Swapped event in `src/superstate/Superstate.sol:88` with input as the input amount. When fees are enabled, input holds the net amount forwarded to Superstate after the fee deduction, while the amount pulled from the user is the gross amount. The fee is not included in the event either, so consumers of Swapped alone cannot reconstruct what the user actually paid.

Indexers, analytics, and user-facing transaction history, therefore, understate purchase amounts for every fee-enabled swap. The stored totals in lines 53-65 track `inputSum` and `feeSum` separately, but the event is the canonical interface for off-chain consumers.

Recommendation

We recommend including the gross input amount and the fee in the Swapped event, along with the net input and output amounts.

Status: Resolved

13. Non-standard payment tokens break swap execution and accounting

Severity: Informational

- `src/superstate/Superstate.sol:32`
- `src/superstate/Superstate.sol:45-47`

The swap function in `src/superstate/Superstate.sol:32` accepts an arbitrary token address and assumes that a successful `transferFrom` in lines 45-47 delivers exactly `input` tokens to the adapter. For fee-on-transfer or rebasing tokens, the adapter receives less than `input` while still approving and forwarding the full amount to `buyTheDip` in line 76 and recording `input` in its totals.

The purchase then either reverts due to the balance shortfall, or it consumes other balances held by the adapter, for example, funds left behind by earlier partial fills, to cover the difference. [Superstate's documentation](#) identifies USDC as the payment token, so the generic parameter

adds risk without a matching requirement.

Recommendation

We recommend storing the supported payment token address and rejecting any other input token. If multiple payment tokens must be supported, we recommend measuring the adapter's balance before and after the `transferFrom` call and working with the amount actually received.

Status: Resolved

14. Swap fee can be set to 100 percent

Severity: Informational

- `src/swap/TokenSwap.sol:79-84`
- `src/superstate/Superstate.sol:41`
- `src/superstate/Superstate.sol:50`
- `src/superstate/Superstate.sol:68`
- `src/superstate/Superstate.sol:76`

`setBps` allows any value up to and including `ONE_HUNDRED_P`. Since `ONE_HUNDRED_P` is 10000 basis points, the owner can configure a 100 percent fee. With `bps` set to 10000, the fee consumes the entire previewed input, leaving zero net payment for `buyTheDip`.

For users following Superstate guidance and setting a nonzero `slip` from a preview quote, this causes `buyTheDip` to receive a zero amount and revert with `InsufficientOutput`. All swap calls remain broken until `bps` is lowered. If a user supplies `slip` as zero, execution can succeed, but the adapter keeps the full payment as fees and returns no equity tokens.

Recommendation

We recommend changing the `setBps` validation so `points` must be lower than `ONE_HUNDRED_P`, not merely lower than or equal to it. The protocol should also consider a much smaller hard cap, such as 10 percent, so destructive fee settings are impossible even below the 100 percent boundary.

Status: Resolved